Przetwarzanie Równoległe — Projekt GPU
Dodawanie wartości macierzy w wyznaczonym promieniu

Sprawozdanie laboratoryjne

Daniel Igansiak 151238

Data oddania: 11 stycznia 2026

1 Opis użytej karty graficznej

Testy wykonano na karcie **NVIDIA GeForce GTX 1080**, opartej na układzie **GP104** w architekturze **Pascal**. Karta posiada **Compute Capability 6.1**, co określa zestaw dostępnych instrukcji, ograniczeń i parametrów sprzętowych.

Parametry architektury

Pozycja	Wartość
Model GPU	NVIDIA GeForce GTX 1080
Układ scalony	GP104-400
Architektura	Pascal
Compute Capability	6.1
Liczba SM	20
CUDA Cores	2560 (128 na SM)
Jednostki SP (FP32)	2560
Efektywne jednostki DP (FP64)	80 (1/32 SP)
Jednostki całkowitoliczbowe (INT32)	2560 (współdzielone z FP32)
Jednostki SFU	80 (4 na SM)
Jednostki teksturujące (TMU)	160 (8 na SM)
Pamięć globalna (VRAM)	8 GB GDDR5X
Przepustowość pamięci	320 GB/s
L2 cache	2 MB
Shared memory / SM	96 KB (konfigurowalne)
Rejestry / SM	65 536 (32-bit)
Maks. wątków / SM	2048
Rozmiar Osnowy (Warp)	32

Ograniczenia CC 6.1

- ograniczona wydajność FP64 (1/32 FP32),
- maksymalnie 48 KB pamięci współdzielonej na blok,
- maksymalnie 1024 wątki na blok,
- 64 warpy aktywne na SM,
- współdzielone jednostki INT32/FP32.

2 Zakres zrealizowanego zadania

Rozważany problem obliczeniowy dotyczy dwuwymiarowej tablicy wejściowej `TAB[N*N]`, w której elementy kolejnych wierszy są rozmieszczone w pamięci ciągłej (element `TAB[i][j]` znajduje się pod adresem `TAB[i*N + j]`). Dla zadanej wartości promienia R należy obliczyć tablicę wynikową `OUT[(N-2R)*(N-2R)]`, w której każdy element stanowi sumę wszystkich wartości z kwadratowego sąsiedztwa o boku $(2R + 1)$, obejmującego dany punkt tablicy wejściowej. Oznacza to, że dla każdego elementu wyjściowego wykonywana jest suma $(2R + 1)^2$ elementów wejściowych. Przykładowo, dla $R = 1$ obliczenie wartości `OUT[i-R][j-R]` wymaga zsumowania dziewięciu elementów otoczenia `TAB[i][j]`.

Celem projektu było:

Porównanie jakości przetwarzania czterech wariantów kerneli CUDA realizujących sumowanie wartości w oknie o promieniu R:

- **Kernel A** – dostęp do pamięci globalnej, w pełni koalescowany,
- **Kernel B** – dostęp do pamięci globalnej, celowo rozkoalescowany,
- **Kernel C** – użycie pamięci współdzielonej (shared memory), dostęp koalescowany,
- **Kernel D** – użycie pamięci współdzielonej z transpozycją, generujący konflikty banków.

Zmierzyć wpływ: wielkości bloku wątków BS, rodzaju kernela oraz rozmiaru macierzy N na czas wykonania kernela, GFLOPS, estymowaną przepustowość pamięci i arithmetic intensity (AI).

3 Kluczowe fragmenty kodu i ich omówienie

3.1 Kernel A – dostęp koalescjonowany

```
int outX0 = (bx * BLOCK_SIZE + tx) * k;
int outY  =  by * BLOCK_SIZE + ty;

for (int t = 0; t < k; ++t) {
    int outX = outX0 + t;
    int inXcenter = outX + R;
    int inYcenter = outY + R;

    float sum = 0.0f;

    for (int dy = -R; dy <= R; ++dy) {
        int rowBase = (inYcenter + dy) * N;
        for (int dx = -R; dx <= R; ++dx) {
            sum += A[rowBase + (inXcenter + dx)];
        }
    }

    C[outY * outWidth + outX] = sum;
}
```

Wyjaśnienie:

- wyliczenie `outX0`, `outY` mapuje wątek bloku do globalnych współrzędnych wyjścia; mnożenie przez `k` oznacza że każdy wątek liczy `k` kolejnych wyjść w osi X,
- pętle po `dy`, `dx` czytają elementy rzędu `A[rowBase + x]` – gdy `tx` sąsiaduje w warpie, odczyty po kolumnie są liniowe - koalescencja (warps czytają kolejne słowa pamięci),
- brak użycia pamięci współdzielonej: brak lokalnego bufora powoduje wielokrotne odczyty z globalnej pamięci (wysoki ruch DRAM),
- ograniczenie wydajności: memory-bound - przepustowość globalnej pamięci jest wąskim gardłem.

3.2 Kernel B – permutacja `tx` (celowe niekoalescjonowanie)

```
const int half = BLOCK_SIZE / 2;
int perm_tx = (tx % 2 == 0) ? (tx / 2) : (half + tx / 2);

int outX0 = (bx * BLOCK_SIZE + perm_tx) * k;
int outY  =  by * BLOCK_SIZE + ty;
```

Wyjaśnienie:

- `perm_tx` permutuje indeksy wątków wewnątrz bloku (np. 0,1,2,3 - 0,2,1,3), co powoduje że kolejne wątki warpu nie czytają sąsiednich adresów w pamięci globalnej,

- odczyty elementów A pozostają semantycznie takie same, ale ułożenie warpu w pamięci jest rozproszone - brak koalescencji, więcej transakcji DRAM i niższe wykorzystanie przepustowości,
- użyte jako demonstracja wpływu wzorca dostępu na wydajność przy zachowaniu poprawności obliczeń.

3.3 Zmienne indeksujące i znaczenie

```
int bx = blockIdx.x;   // indeks bloku w X
int by = blockIdx.y;   // indeks bloku w Y
int tx = threadIdx.x;  // indeks wątku w bloku (X)
int ty = threadIdx.y;  // indeks wątku w bloku (Y)

int outWidth  = N - 2 * R;
int outHeight = N - 2 * R;
int k = ...;           // liczba wyjść liczonych przez wątek w X
int R = ...;           // promień okna
```

Wyjaśnienie:

- `bx,by,tx,ty` definiują mapowanie obliczeń z przestrzeni bloków/wątków na współrzędne macierzy,
- `outWidth/outHeight` to rozmiary macierzy wyjściowej (odjęcie halo 2R),
- `k` zwiększa liczbę wyjść per thread (amortyzacja overheadu indeksowania, wpływa na tileWidth przy shared),
- `R` ustala rozmiar okna: W = $(2R+1)^2$; $dueRzwikszareuse, aletezwikszawymgsharedmemorydlati$

3.4 Kernel C – ładowanie tile do shared memory z halo

```
extern __shared__ float tile[]; // 1D buffer

int blockOutX = bx * (BLOCK_SIZE * k);
int blockOutY = by * BLOCK_SIZE;

int tileWidth  = BLOCK_SIZE * k + 2 * R;
int tileHeight = BLOCK_SIZE + 2 * R;
int tileX0 = blockOutX;
int tileY0 = blockOutY;

/* współdzielone wczytywanie: pętle z krokiem BLOCK_SIZE */
for (int dy = ty; dy < tileHeight; dy += BLOCK_SIZE) {
    int globalY = tileY0 + dy;
    int base = dy * tileWidth;
    for (int dx = tx; dx < tileWidth; dx += BLOCK_SIZE) {
        int globalX = tileX0 + dx;
```

```
        tile[base + dx] = A[globalY * N + globalX];
    }
}
__syncthreads();

/* odczyt z tile: lokalne współrzędne (lx,ly) z offsetem R */
int lx = tx * k + t + R;
int ly = ty + R;
for (int dy = -R; dy <= R; ++dy)
  for (int dx = -R; dx <= R; ++dx)
    sum += tile[(ly + dy) * tileWidth + (lx + dx)];
```

Wyjaśnienie:

- deklaracja `extern __shared__ float tile[]` używa dynamicznej pamięci współdzielonej przekazanej przy wywołaniu kernela,
- `tileWidth/tileHeight` uwzględniają halo (2R), aby wszystkie potrzebne elementy do okien mogli być dostępne lokalnie w shared,
- wczytywanie wykonywane jest przez wszystkie wątki bloku: pętle z krokiem `BLOCK_SIZE` gwarantują, że każdy wątek bierze część elementów tile (współbieżne, zrównoważone),
- indeksowanie do shared jako `base + dx` daje liniowy, spójny layout w shared - odczyty później są szybkie i bez konfliktów (przy odpowiednim rozmiarze banków),
- `__syncthreads()` synchronizuje wątki przed użyciem tile - zapobiega hazardom pamięciowym,
- korzyść: znaczące zredukowanie liczby odczytów z pamięci globalnej (większe AI i wyższe GFLOPS).

3.5 Kernel D – transponowany zapis do shared (powód konfliktów banków)

```
/* store transposed: logical (dy,dx) stored at index (dx * tileHeight + dy) */
tile[dx * tileHeight + dy] = A[globalY * N + globalX];

...
/* read logical (ly+dy, lx+dx) from tile[(lx+dx) * tileHeight + (ly+dy)] */
sum += tile[(lx + dx) * tileHeight + (ly + dy)];
```

Wyjaśnienie:

- zapis transponowany zmienia fizyczne rozmieszczenie logicznego wiersz×kolumna w shared (column-major w 1D indeksowaniu),
- w architekturze shared memory istnieją banki (np. 32) - jednoczesne odwołania wielu wątków do elementów należących do tego samego banku powodują bank conflicts - sekwencyjne serwowanie transakcji - spadek przepustowości shared,

- transponowany layout może prowadzić do wielu konfliktów przy typowym odczycie logicznym (wierszowym) wykonywanym przez wątki; efekt pogłębia się dla większych tile i większego BS.

3.6 Obliczanie wymaganego shared memory i fallback w dispatcherze

```
int tileWidth  = BLOCK_SIZE * k + 2 * R;
int tileHeight = BLOCK_SIZE + 2 * R;
size_t sharedBytes = (size_t)tileWidth * (size_t)tileHeight * sizeof(float);

if (sharedBytes > sharedLimit) {
    // fallback do kernel A (global memory) - sharedBytes = 0
    actualMode = 'a';
}
...
MatrixAddRadCUDA_C<BLOCK_SIZE><<<grid, threads, sharedBytes, stream>>>(...);
```

Wyjaśnienie:

- obliczana jest liczba bajtów potrzebnych do tile w shared (liczba elementów $\times$ 4 B),
- jeśli wymóg przekracza fizyczny limit shared per block (odczytany z `cudaGetDeviceProperties`), kod wykonuje fallback do wersji globalnej by uniknąć błędu launchu,
- dispatcher wstrzykuje `sharedBytes` jako parametr launchu kernela (third execution configuration argument).

3.7 Konfiguracja siatki i wątków (grid / block / k)

```
dim3 threads(BLOCK_SIZE, BLOCK_SIZE);

int gridDimX = (outWidth  + BLOCK_SIZE * k - 1) / (BLOCK_SIZE * k);
int gridDimY = (outHeight + BLOCK_SIZE     - 1) /  BLOCK_SIZE;
dim3 grid(gridDimX, gridDimY);
```

Wyjaśnienie:

- `threads` = (BLOCK_SIZE, BLOCK_SIZE) - blok ma `BLOCK_SIZE`$\times$`BLOCK_SIZE` wątków,
- w osi X każdy blok pokrywa `BLOCK_SIZE * k` kolumn wyjścia (bo każdy wątek liczy `k` wyjść),
- `gridDimX/gridDimY` obliczane są jako pokrycie całej macierzy wyjściowej z uwzględnieniem zaokrąglenia w górę,
- wpływ `k`: większe `k` zmniejsza liczbę bloków w X i zwiększa tileWidth (więcej shared memory).

3.8 Pomiar czasu kernela (CUDA events)

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start, stream);
for (int i = 0; i < nIter; ++i)
    LaunchMatrixAddKernel<...>(...);
cudaEventRecord(stop, stream);
cudaEventSynchronize(stop);

float msec = 0.0f;
cudaEventElapsedTime(&msec, start, stop);
float avgKernelMs = msec / (float)nIter;
```

Wyjaśnienie:

- CUDA events dają precyzyjny pomiar czasu GPU (kernel-only), nie zawierający kosztów kopiowania H2D/D2H jeśli używany jest ten sam stream,
- konieczna jest synchronizacja `cudaEventSynchronize(stop)` by mieć poprawny wynik `cudaEventElapsedTime`,
- średni czas per iterację = łączny czas / `nIter` (używane do wyliczenia GFLOPS).

3.9 CPU referencja i walidacja wyników

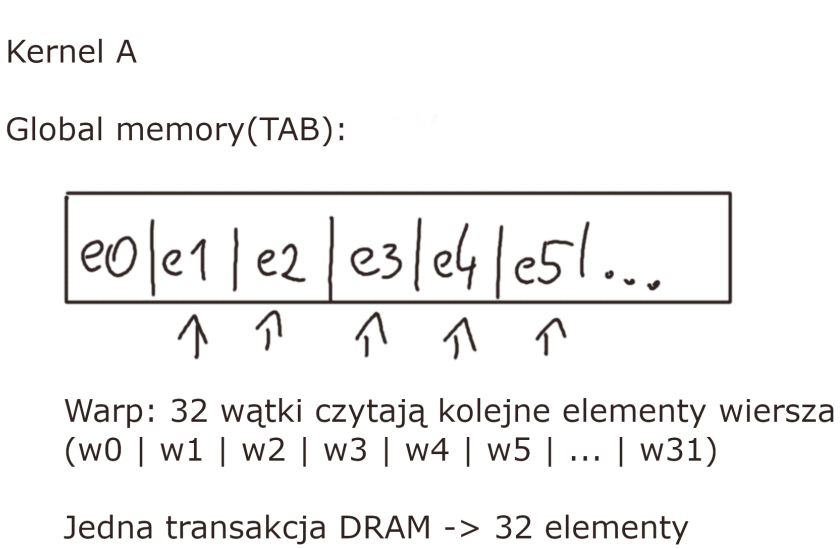
```
void MatrixAddRadCPU(float *C_ref, const float *A, int N, int R) {
    int outWidth  = N - 2 * R;
    int outHeight = N - 2 * R;
    for (int y = 0; y < outHeight; ++y)
      for (int x = 0; x < outWidth; ++x) {
        float sum = 0.0f;
        int inXcenter = x + R;
        int inYcenter = y + R;
        for (int dy = -R; dy <= R; ++dy)
          for (int dx = -R; dx <= R; ++dx)
            sum += A[(inYcenter + dy) * N + (inXcenter + dx)];
        C_ref[y * outWidth + x] = sum;
      }
}

/* walidacja wyników GPU vs CPU */
for (int i = 0; i < outWidth * outHeight; ++i) {
    double diff = fabs(h_C[i] - h_C_ref[i]);
    if (diff > eps && diff > eps * fabs(h_C_ref[i])) { /* FAIL */ }
}
```

Wyjaśnienie:

- implementacja CPU jest prostą, bezoptymalną referencją zapewniającą poprawność semantyczną,
- walidacja porównuje wyniki element-po-elemencie z tolerancją numeryczną (relacyjną i bezwzględną) - wykrywa błędy w wariantach C/D (bank conflicts nie zmieniają wyniku, ale błędna indeksacja już tak),
- przydatne dla debugowania: lokalizowanie pierwszego mismatch i wypisanie sąsiedztwa wartości A/C dla analizy.

4 Rysunki i opis dostępu do pamięci

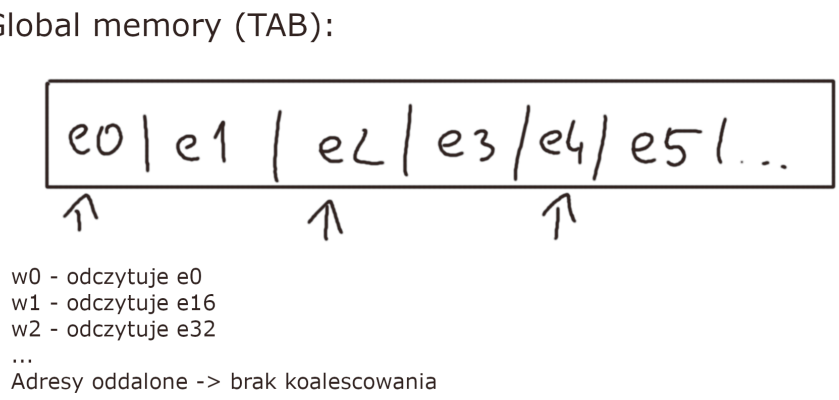
Dostęp koalescowany (Kernel A)



Rysunek 1: Rysunek dostępu koalescowanego

W kernelu A wątki w warpie odczytują kolejne adresy z pamięci globalnej. Adresy są ułożone liniowo, więc kontroler DRAM może obsłużyć cały warp jedną transakcją 128-B. Dostęp jest w pełni koalescowany, co minimalizuje liczbę transakcji i zapewnia najwyższą przepustowość. Brak dodatkowych opóźnień – każdy element jest pobierany tylko raz.

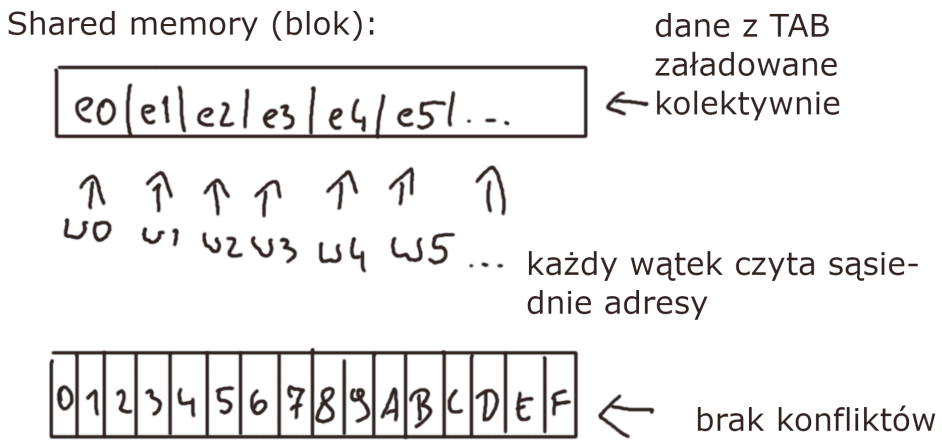
Dostęp niekoalescowany (Kernel B)



Rysunek 2: Rysunek dostępu niekoalescowanego

W kernelu B wątki w warpie odczytują dane z odległych adresów, np. co kilkanaście lub kilkadziesiąt elementów. Kontroler DRAM nie może połączyć tych dostępów w jedną transakcję, więc każdy wątek generuje osobną transakcję. Powoduje to: drastyczny wzrost liczby transakcji DRAM, niską efektywność przepustowości, dominację czasu dostępu nad obliczeniami. Dostęp jest niekoalescowany, co jest najgorszym możliwym scenariuszem dla pamięci globalnej.

Shared memory (Kernel C)

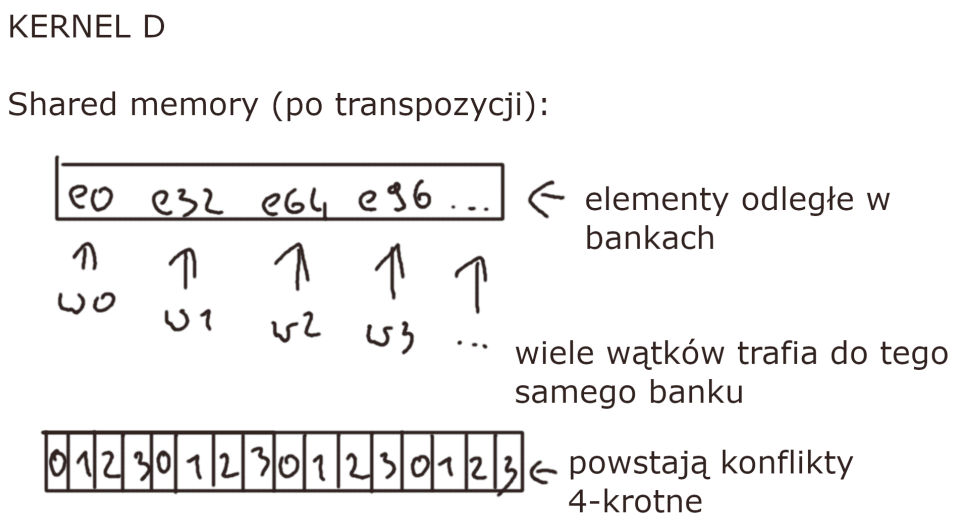


Rysunek 3: Rysunek pamięci współdzielonej bez konfliktów

Kernel C przenosi dane z pamięci globalnej do shared memory, wykorzystując wszystkie wątki bloku. Dane są ułożone liniowo, więc każdy wątek trafia do innego banku pamię-

ci współdzielonej. Brak konfliktów banków oznacza: każdy dostęp jest wykonywany w jednym cyklu, dane są używane wielokrotnie bez ponownego czytania z DRAM, przepustowość shared memory jest maksymalna. Dostęp jest efektywny, a ruch do pamięci globalnej minimalny.

Shared memory z konfliktami (Kernel D)



Rysunek 4: Rysunek pamięci współdzielonej z konfliktami

Kernel D również korzysta ze shared memory, ale sposób indeksowania powoduje, że wątki w warpie odwołują się do tych samych banków. Przykład: transpozycja lub dostęp kolumnowy. Konsekwencje: banki muszą serializować dostęp, opóźnienia rosną proporcjonalnie do liczby konfliktów, mimo użycia shared memory wydajność spada nawet kilkukrotnie. Dostęp jest nieefektywny, ponieważ występują konflikty banków pamięci współdzielonej.

5 Wzory użyte do obliczeń

GFLOPS

$$\text{GFLOPS} = \frac{\text{ops} \cdot 10^{-9}}{t_{\text{kernel}}}$$

Liczba operacji

$$\text{ops} = (N - 2R)^2 \cdot (2R + 1)^2$$

Intensywność arytmetyczna

$$AI = \frac{\text{liczba FLOP}}{\text{liczba bajtów odczytanych z DRAM}}$$

6 Wyniki pomiarów

6.1 Różne pamięci i efektywność dostępu.

Tabela 1: Porównanie wariantów A/B/C/D (BS=8, N=1024, R=20, k=2)

Kernel	t_{kernel} [ms]	GFLOPS	AI [FLOP/B]	BW_est [GB/s]	shared [B]
A	16.929	96.146	0.2499	384.813	0
B	14.152	115.014	0.2499	460.330	0
C	2.858	569.445	18.955	30.042	10 752
D	11.329	143.670	18.955	7.579	10 752

Tabela 2: Porównanie wariantów A/B/C/D (BS=16, N=1024, R=20, k=2)

Kernel	t_{kernel} [ms]	GFLOPS	AI [FLOP/B]	BW_est [GB/s]	shared [B]
A	10.537	154.463	0.2499	618.221	0
B	10.158	160.236	0.2499	641.324	0
C	2.890	563.245	46.676	12.067	16 128
D	11.272	144.393	46.676	3.094	16 128

Tabela 3: Porównanie wariantów A/B/C/D (BS=32, N=1024, R=20, k=2)

Kernel	t_{kernel} [ms]	GFLOPS	AI [FLOP/B]	BW_est [GB/s]	shared [B]
A	9.820	165.753	0.2499	663.407	0
B	10.130	160.673	0.2499	643.074	0
C	2.923	556.789	86.904	6.407	29 952
D	22.882	71.133	86.904	0.819	29 952

6.2 Wpływ wielkości bloku BS.

Tabela 4: Wpływ BS na wydajność kernela A (N=1024, R=20, k=2)

BS	t_{kernel} [ms]	GFLOPS	ops [$\cdot 10^6$]	BW_est [GB/s]	AI [FLOP/B]
8	16.929	96.146	78.43	384.81	0.2499
16	10.537	154.463	1 627.64	618.22	0.2499
32	9.820	165.753	1 627.64	663.41	0.2499

Tabela 5: Wpływ BS na wydajność kernela B (N=1024, R=20, k=2)

BS	t_{kernel} [ms]	GFLOPS	ops [$\cdot 10^6$]	BW_est [GB/s]	AI [FLOP/B]
8	14.152	115.014	78.43	460.33	0.2499
16	10.158	160.236	1 627.64	641.32	0.2499
32	10.130	160.673	1 627.64	643.07	0.2499

Tabela 6: Wpływ BS na wydajność kernela C (N=1024, R=20, k=2)

BS	t_{kernel} [ms]	GFLOPS	ops [$\cdot 10^6$]	BW_est [GB/s]	AI [FLOP/B]
8	2.858	569.445	1 627.64	30.042	18.955
16	2.890	563.245	1 627.64	12.067	46.676
32	2.923	556.789	1 627.64	6.407	86.904

Tabela 7: Wpływ BS na wydajność kernela D (N=1024, R=20, k=2)

BS	t_{kernel} [ms]	GFLOPS	ops [$\cdot 10^6$]	BW_est [GB/s]	AI [FLOP/B]
8	11.329	143.670	1 627.64	7.579	18.955
16	11.272	144.393	1 627.64	3.094	46.676
32	22.882	71.133	1 627.64	0.819	86.904

6.3 Wpływ rozmiaru macierzy N na wysycanie GPU.

Tabela 8: Wpływ rozmiaru N na wydajność kernela A (BS=16, R=20, k=2)

N	t_{kernel} [ms]	GFLOPS	ops [$\cdot 10^6$]	BW_est [GB/s]	AI [FLOP/B]
256	0.542	144.765	78.43	579.403	0.2499
512	2.485	150.699	374.50	603.154	0.2499
1024	10.537	154.463	1 627.64	618.221	0.2499
2048	44.566	152.086	6 777.90	608.705	0.2499
4096	197.114	140.296	27 654.36	561.519	0.2499

Tabela 9: Wpływ rozmiaru N na wydajność kernela B (BS=16, R=20, k=2)

N	t_{kernel} [ms]	GFLOPS	ops [$\cdot 10^6$]	BW_est [GB/s]	AI [FLOP/B]
256	0.538	145.744	78.43	583.322	0.2499
512	2.418	154.871	374.50	619.851	0.2499
1024	10.158	160.236	1 627.64	641.324	0.2499
2048	42.441	159.701	6 777.90	639.185	0.2499
4096	185.865	148.787	27 654.36	595.504	0.2499

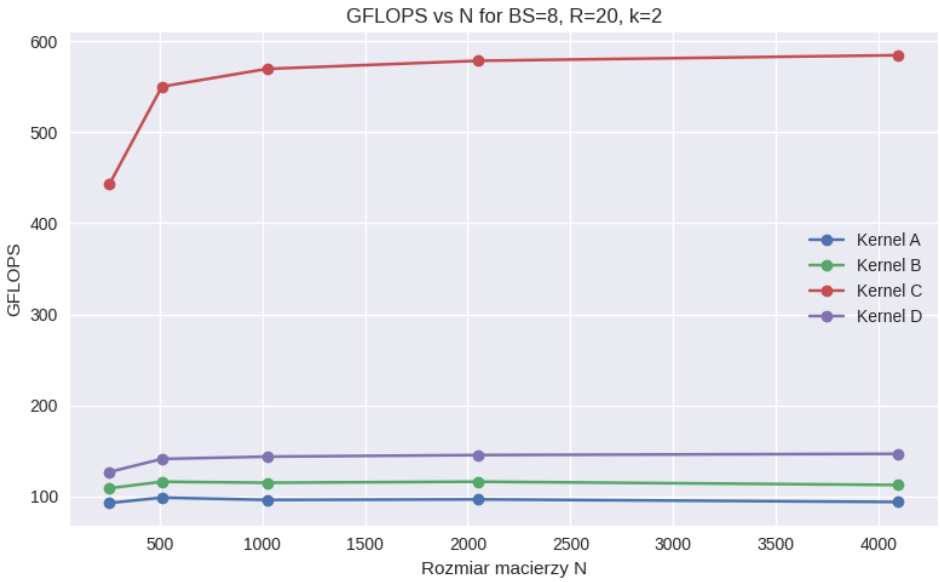
Tabela 10: Wpływ rozmiaru macierzy N na wydajność kernela C (BS=16, R=20, k=2)

N	t_{kernel} [ms]	GFLOPS	ops [$\cdot 10^6$]	BW_est [GB/s]	AI [FLOP/B]
256	0.158	496.394	78.43	11.185	44.382
512	0.694	539.446	374.50	11.738	45.958
1024	2.890	563.245	1 627.64	12.067	46.676
2048	11.745	577.069	6 777.90	12.273	47.019
4096	47.975	576.432	27 654.36	12.216	47.187

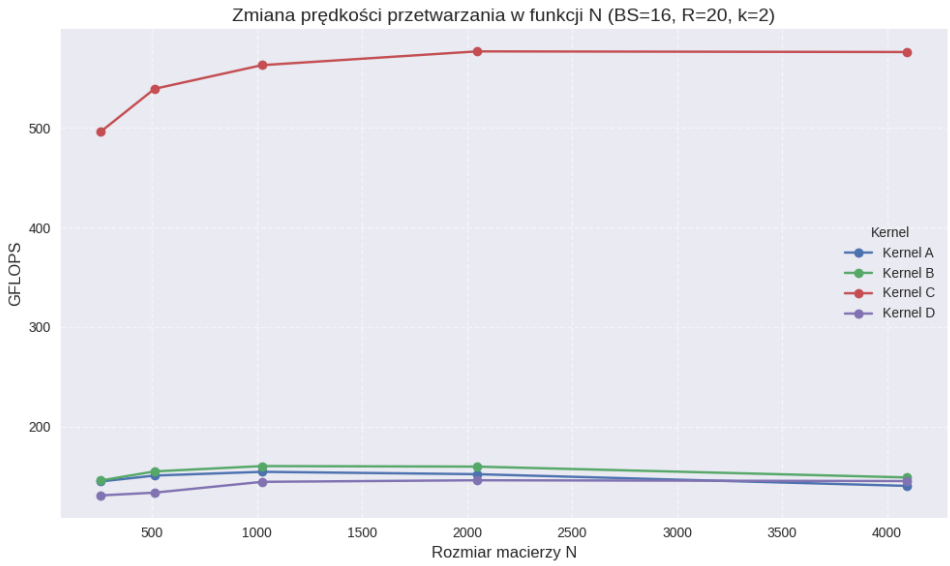
Tabela 11: Wpływ rozmiaru macierzy N na wydajność kernela D (BS=16, R=20, k=2)

N	t_{kernel} [ms]	GFLOPS	ops [$\cdot 10^6$]	BW_est [GB/s]	AI [FLOP/B]
256	0.600	130.760	78.43	2.946	44.382
512	2.805	133.519	374.50	2.905	45.958
1024	11.272	144.393	1 627.64	3.094	46.676
2048	46.436	145.962	6 777.90	3.104	47.019
4096	190.498	145.169	27 654.36	3.076	47.187

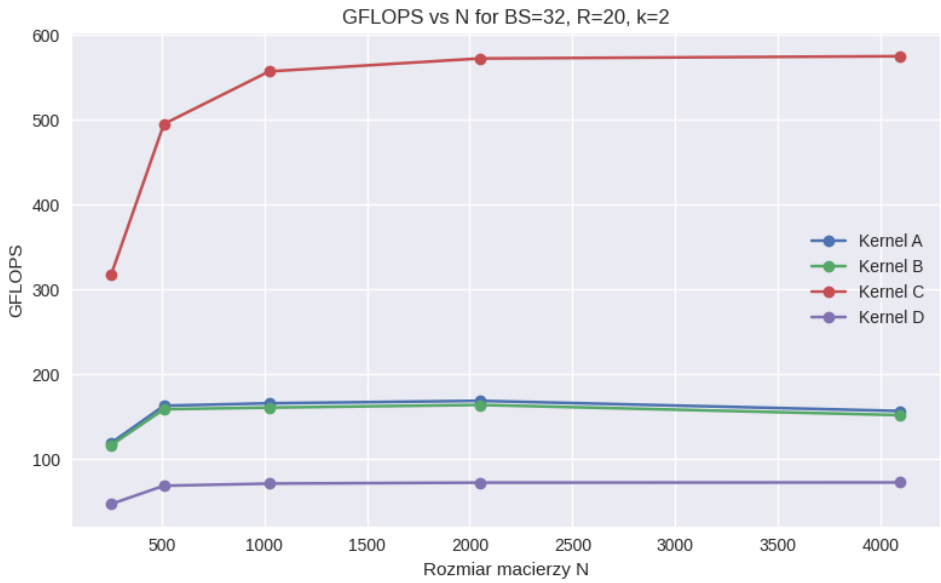
7 Wykresy prędkości przetwarzania



Rysunek 5: Zależność prędkości przetwarzania od rozmiaru instancji problemu dla BS = 8, R = 20, k = 2



Rysunek 6: Zależność prędkości przetwarzania od rozmiaru instancji problemu dla BS = 16, R = 20, k = 2



Rysunek 7: Zależność prędkości przetwarzania od rozmiaru instancji problemu dla BS = 32, R = 20, k = 2

8 Wnioski

- **Kernel A osiąga stabilną i przewidywalną wydajność**, ponieważ wykorzystuje w pełni koalescowany dostęp do pamięci globalnej. W architekturze GPU (Pascal, CC 6.1) kontroler pamięci łączy 32 odczyty warpu w jedną transakcję DRAM, co minimalizuje liczbę operacji na magistrali. Wydajność jest ograniczona przepustowością pamięci globalnej (memory-bound), ponieważ każdy element okna jest pobierany z DRAM bez ponownego użycia.

- **Kernel B jest wolniejszy od A mimo identycznej liczby operacji**, ponieważ brak koalescencji powoduje, że każdy wątek warpu generuje osobną transakcję DRAM. W efekcie liczba transakcji rośnie nawet 32-krotnie, co prowadzi do przeciążenia kontrolera pamięci i drastycznego spadku efektywnej przepustowości. To pokazuje, że wzorzec dostępu ma większy wpływ na wydajność niż sama liczba operacji arytmetycznych.

- **Kernel C jest zdecydowanie najszybszy**, ponieważ przenosi dane do pamięci współdzielonej (shared memory), gdzie opóźnienia dostępu są rzędu kilkudziesięciu cykli krótsze niż w DRAM. Każdy element okna jest ładowany tylko raz, a następnie wielokrotnie wykorzystywany przez wątki bloku. Zwiększa to intensywność arytmetyczną (AI) nawet kilkudziesięciokrotnie. Brak konfliktów banków powoduje, że dostęp do shared memory jest w pełni równoległy, a SM osiągają wysokie wykorzystanie jednostek obliczeniowych.

- **Kernel D jest wolniejszy od C mimo identycznej liczby operacji i identycznego reuse danych**, ponieważ sposób indeksowania powoduje konflikty banków pamięci współdzielonej. W architekturze Pascal shared memory jest podzielona na 32 banki; jeśli wiele wątków warpu odwołuje się do tego samego banku, dostęp musi zostać zserializowany. W kernelu D konflikty te występują systematycznie (np. przy transpozycji), co prowadzi do liniowego wzrostu opóźnień i spadku przepustowości shared memory.

- **Wzrost rozmiaru macierzy N powoduje wzrost GFLOPS aż do osiągnięcia wysycenia GPU**. Dla małych N liczba aktywnych bloków jest niewystarczająca, aby w pełni wypełnić wszystkie SM - GPU pracuje w trybie underutilized. Dopiero po przekroczeniu pewnego progu (N_{wys}) liczba bloków jest na tyle duża, że scheduler może stale utrzymywać pełne obciążenie SM, maskując opóźnienia pamięci i osiągając maksymalną przepustowość obliczeniową.

- **Po osiągnięciu wysycenia dalszy wzrost N nie zwiększa prędkości**, ponieważ GPU osiąga limit przepustowości pamięci (dla A/B) lub limit przepustowości shared memory / ALU (dla C/D). Zgodnie z modelem Roofline, dalsze zwiększanie liczby operacji nie poprawia GFLOPS, jeśli osiągnięto już granicę przepustowości lub intensywności arytmetycznej.

- **Wzrost rozmiaru bloku BS wpływa na wydajność w sposób nieliniowy**. Dla kerneli A/B większy blok poprawia koalescencję i zwiększa liczbę aktywnych wątków na SM, co poprawia maskowanie opóźnień. Dla kerneli C/D większy blok oznacza większy kafel w shared memory, co zwiększa reuse danych, ale jednocześnie zwiększa zużycie shared memory per block. To może zmniejszyć occupancy (mniej bloków mieści się na SM), co tłumaczy spadek wydajności dla BS=32 w kernelu D.

- **Parametr k zwiększa intensywność arytmetyczną**, ponieważ każdy wątek wykonuje więcej obliczeń na jednym zestawie danych. Zmniejsza to liczbę odczytów globalnych per wynik i poprawia stosunek FLOP/B. Jednak duże k zwiększa rozmiar kafla w shared memory, co może zmniejszyć occupancy i prowadzić do spadku wydajności dla kerneli C/D przy dużych
- **Kernel C najlepiej skaluje się z parametrami N,BS i k**, ponieważ shared memory pozwala na maksymalizację reuse danych i minimalizację ruchu DRAM. Kernel D pokazuje, że nawet niewielka zmiana sposobu indeksowania może zniszczyć równoległość dostępu i obniżyć wydajność o rząd wielkości.

9 Bibliografia

- https://vgamuseum.info/images/doc/nvidia/gtx1080_whitepaper.pdf
- prezentacje z wykładów Przetwarzania Równoległego